Vectors, functions, and unit tests

Motivating example

What do you think will be the output of this code?

```
1 x <- c(1, 2, 3, 4)
2 y <- c(0, 1)
3
4 x + y
```

Motivating example

```r
x <- c(1, 2, 3, 4)
y <- c(0, 1)

x + y
```

```
[1] 1 3 3 5
```

What has happened here?

Vector recycling

Vector recycling: Repeating elements of the shorter vector to match the length of the longer vector

```
1 x <- c(1, 2, 3, 4)
2 y <- c(0, 1)
3
4 x + y
```

```
[1] 1 3 3 5
```

Vector recycling

```r
x <- c(1, 2, 3)
y <- c(0, 1)

x + y
```

```
Warning in x + y: longer object length is not a multiple of shorter
object
length
```

```
[1] 1 3 3
```

If the length of the longer vector is not a multiple of the length of the shorter vector, R will give a warning – but will still produce output!

Huber loss

```r
huber_loss <- function(y, y_hat, gamma){
  res <- y - y_hat
  ifelse(abs(res) < gamma,
         0.5*res^2,
         gamma*abs(res) - 0.5*gamma^2)
}

huber_loss(y = c(2, 2, 3, 4), y_hat = c(1.5, 2.5), gamma = 1)
```

```
[1] 0.125 0.125 1.000 1.000
```

- How would I modify my function to ensure `y` and `y_hat` have the same length?
- The current code runs fine, even if the output is not what we want. How would I catch this bug in my code?

Error handling

```r
huber_loss <- function(y, y_hat, gamma){
  if(length(y) != length(y_hat)){
    stop("y and y_hat must have the same length")
  }

  res <- y - y_hat
  ifelse(abs(res) < gamma,
         0.5*res^2,
         gamma*abs(res) - 0.5*gamma^2)
}

huber_loss(y = c(2, 2, 3, 4), y_hat = c(1.5, 2.5), gamma = 1)
```

```
Error in huber_loss(y = c(2, 2, 3, 4), y_hat = c(1.5, 2.5), gamma = 1):
y and y_hat must have the same length
```

Finding bugs: unit testing!

```
1 library(testthat)
```

```
1 test_that("Huber loss length error", {
2   expect_error(huber_loss(y = c(2, 2, 3, 4),
3                           y_hat = c(1.5, 2.5), gamma = 1))
4 })
```

```
Test passed with 1 success 😸.
```

Finding bugs: unit testing!

Let's also check that we get an error when `y` is the shorter vector:

```
library(testthat)
```

```
test_that("Huber loss length error", {
  expect_error(huber_loss(y = c(2, 2, 3, 4),
                          y_hat = c(1.5, 2.5), gamma = 1))
  expect_error(huber_loss(y = c(1, 2),
                          y_hat = c(1, 2, 3), gamma = 1))
})
```

```
Test passed with 2 successes 🥳.
```

Finding bugs: unit testing!

We can also test that our function produces the desired values:

$$\rho_\gamma(Y_i, \widehat{Y}_i) = \begin{cases} \frac{1}{2}(Y_i - \widehat{Y}_i)^2 & |Y_i - \widehat{Y}_i| < \gamma \\ \gamma|Y_i - \widehat{Y}_i| - \frac{1}{2}\gamma^2 & |Y_i - \widehat{Y}_i| \geq \gamma \end{cases}$$

```
1 test_that("Huber loss value test 1", {
2   expect_equal(huber_loss(y = 1, y_hat = 0.5, gamma = 1),
3                0.125)
4 })
```

```
Test passed with 1 success 🎉.
```

Finding bugs: unit testing!

This implementation has a bug, but our test passes:

```r
huber_loss <- function(y, y_hat, gamma){
  if(length(y) != length(y_hat)){
    stop("y and y_hat must have the same length")
  }

  res <- y - y_hat
  ifelse(res < gamma,
         0.5*res^2,
         gamma*abs(res) - 0.5*gamma^2)
}
```

```r
test_that("Huber loss value test 1", {
  expect_equal(huber_loss(y = 1, y_hat = 0.5, gamma = 1),
               0.125)
})
```

```
Test passed with 1 success 🌈.
```

Finding bugs: unit testing!

```r
huber_loss <- function(y, y_hat, gamma){
  if(length(y) != length(y_hat)){
    stop("y and y_hat must have the same length")
  }

  res <- y - y_hat
  ifelse(res < gamma,  0.5*res^2, gamma*abs(res) - 0.5*gamma^2)
}
```

```r
test_that("Huber loss value test 1", {
  expect_equal(huber_loss(y = 1, y_hat = 0.5, gamma = 1),
               0.125)
  expect_equal(huber_loss(y = 1, y_hat = 3, gamma = 1),
               1.5)
})
```

```
── Failure: Huber loss value test 1 ────────────────────────────────────────────
Expected `huber_loss(y = 1, y_hat = 3, gamma = 1)` to equal 1.5.
Differences:
```

```
1/1 mismatches
[1] 2 - 1.5 == 0.5

Error:
! Test failed with 1 failure and 1 success.
```

Class activity

Work on the class activity (course website).